

Bridging Misconception Detection and Dialogue-Based Tutoring for Enhanced Learning in Programming Education

Anonymous ACL submission

Abstract

Students learning programming often develop misconceptions that make learning difficult. While recent work shows that large language models (LLMs) can reduce instructional workload (Tian et al., 2025), it remains unclear whether they effectively support student learning. Current AI tutoring systems do not fully address this issue: some provide step-by-step guidance, while others detect mistakes in code, but they do not combine both in an effective way. In this work, we combine two existing approaches: misconception detection (Al-Hossami and Bunescu, 2026) and dialogue-based tutoring (Wang et al., 2025). Our system first identifies misconceptions during the dialogue with the tutor, and then guides students through specific instructions to help them understand and correct their mistakes. Our work highlights the potential of integrating misconception detection with dialogue-based tutoring to better support conceptual understanding in programming education. By focusing on helping students recognize and correct their own misunderstandings, our approach aims to provide a more effective alternative to answer-driven tutoring systems.¹

1 Introduction

The role of LLMs in education has shifted from something that needs to be restricted to something that should be integrated. Recent work shows that LLMs can meaningfully reduce teacher workload (Tian et al., 2025), but whether students actually learn from conversations led by an LLM tutor remains an open question (Francis et al., 2025). This question is especially pressing in introductory programming courses, where students frequently develop persistent misconceptions—false beliefs about language semantics that cause bugs, hinder progress on related concepts, and accumulate throughout a course (Al-Hossami and Bunescu,

2026). Educators do not want students to bypass their misunderstandings by receiving direct answers from generative AI, as this prevents them from building a solid foundation.

Existing LLM-based tutoring systems for coding largely sidestep this problem. Dialogue-based tutors such as TRAVER (Wang et al., 2025) model a student’s knowledge state as a set of missing knowledge components and guide them toward filling those gaps. However, a missing concept and an actively incorrect belief are different failure modes: a student who believes $range(n)$ produces values from 1 to n will not be helped by being told the correct steps. Instead, they need the misconception itself surfaced and corrected. Conversely, McMiner (Al-Hossami and Bunescu, 2026) can mine misconceptions from student code, but it offers no pedagogical path afterwards once one misconception is found.

Therefore, we present our combination of the misconception detection tool (Al-Hossami and Bunescu, 2026) with the dialogue-based tutor system (Wang et al., 2025) specifically for students who are learning how to code. Our workflow aims to guide students to identify their own misconceptions, leading them to recognize the false belief before progressing toward task completion.

2 Related Work

2.1 Dialogue-Based Task Tutoring with LLMs

LLM-powered tutoring agents have been explored across many educational domains, including language learning (Kwon et al., 2024), math (Wang et al., 2024), and science (Zhang et al., 2024). However, most of this work targets knowledge delivery that helps students understand a specific concept, and evaluates success through closed-form assessments such as question answering or multiple-choice tests.

Coding tutoring is a harder regime: tasks are

¹Github: [Link](#), Hugging Face: [Link](#).

open-ended, require understanding cross-file dependencies, and admit many valid solutions, thus making it harder to evaluate (Wang et al., 2025). Early LLM-based code tutors such as CHAT-TUTOR (Chen et al., 2024) primarily plays a reactive role, answering student questions or clarifying doubts on demand. TreeInstruct (Zhao et al., 2024) moves toward proactive guidance using Socratic tree-based questioning for code debugging, but does not explicitly model task completion as the tutoring objective.

TRAVER (Wang et al., 2025) addresses this by combining knowledge tracing with a turn-by-turn verifier that scores candidate tutor utterances against the likelihood of eventual task completion.

2.2 Misconception Detection in Programming

Identifying where and why students go wrong in code has a long history in CS education research. Early systems such as PROUST (Johnson and Soloway, 1984) and later rule-based tools relied on hand-crafted patterns targeting a small set of known misconceptions. A common limitation is that these approaches either operate over a fixed taxonomy (Brown and Altmirri, 2017), or they simply flag the bug without articulating the underlying false belief.

McMining addresses this gap by framing misconception discovery as a generative task: given a bag of problem-code pairs from a student, an LLM-based miner produces a natural-language description of the misconception the code exhibits, including novel misconceptions outside any predefined bank. We build on McMining as the diagnostic component of our pipeline, using its output as a grounding signal for downstream remediation.

3 Dataset

We evaluate on EvoCodeBench (Li, 2024), a repository-level coding benchmark containing 275 tasks across 25 Python repositories. Each task requires completing a function body given varying levels of context (function signature, dependency paths, or surrounding code). The original TRAVER paper (Wang et al., 2025) filtered this to 100 tasks by excluding functions with zero cross-file dependencies, since those do not exercise the knowledge-tracing component of the tutoring pipeline.

We further restrict to a subset of **22 tasks across 4 repositories**: EasyVolcap (12 tasks, volumetric video capture), searcharray (6 tasks, infor-

mation retrieval), stable-diffusion-webui-forge (3 tasks, image generation), and UHGEval/xinhua (1 task, hallucination evaluation). This reduction was necessary because each repository requires its own virtual environment (~ 5 GB each) with project-specific dependencies. Additionally, EasyVolcap required an isolated pyenv environment on HPC due to its Linux-only CUDA extensions and Python version constraints. The selected repositories span a range of task difficulties, from near-trivial (sd-forge, UHGEval) to extremely challenging (searcharray), providing a representative cross-section of the benchmark.

4 Methodology

We follow the original authors’ approach to run the baseline. However, we made several modifications due to resource constraints.

4.1 Model Constraints

TRAVER used Mixtral-8x7B, which is a Sparse Mixture-of-Experts model, for the student agent during dialogue simulation. However, it is no longer available via hosted inference APIs, and it requires ~ 90 GB of VRAM which exceeds our available compute budget. To avoid potential data contamination with the coding benchmark, the original authors require the student agent’s model to have a release date prior to October 2023. Hence, we substitute Mistral-7B-Instruct-v0.2 for the student dialogue model, as it satisfies this constraint. For posttest code generation, we use Llama-3.1-8B-Instruct via the HuggingFace Inference API with $N=5$ samples, $T=0.4$, and $\text{top}_p = 0.95$. Table 1 summarizes the differences between our setup and the original paper.

4.2 Infrastructure

Due to GPU memory limits, we use a split-compute setup. The tutor model, Llama-3.1-70B-Instruct, runs through the HuggingFace Serverless Inference API, while the student model and verifier run locally in 4-bit NF4 quantization. The verifier is executed in separate subprocesses to reduce CUDA memory issues.

4.3 Implementation Details

Our pipeline combines TRAVER’s tutor–student dialogue framework with McMining’s misconception extraction, with the goal of improving tutoring quality by surfacing student misconceptions and conditioning the tutor on them. The pipeline proceeds

in three stages: baseline dialogue generation, misconception mining, and misconception-conditioned re-tutoring.

Baseline dialogue generation. Following TRAYER, we instantiate a tutor agent (Llama-3.1-70B-Instruct) and a student agent (Mistral-7B-Instruct-v0.2) and run multi-turn tutoring dialogues on each selected EvoCodeBench task. Each dialogue runs for up to 8 rounds, with early stopping when the verifier determines the student has acquired sufficient knowledge. After dialogue completion, we generate posttest prompts for each round by truncating the dialogue to that round’s turns, and use Llama-3.1-8B-Instruct to generate $k=10$ candidate code completions per prompt. We evaluate these against the task’s test suite using $\text{pass}@k$, following the original TRAYER evaluation protocol. This constitutes our baseline.

Misconception analysis. We first applied McMining (Al-Hossami and Bunesco, 2026) to student-generated code from the completed baseline dialogues to assess the prevalence of misconceptions. Using Gemini 2.5 Flash as the mining model, we found that McMiner detected candidate misconceptions in 81.1% of code samples (Section 5.4). This high detection rate raised the question of how to best incorporate misconception information into the tutoring pipeline. Injecting all detected misconceptions at once into the tutor’s prompt risked overwhelming the model with noisy or redundant signals, particularly since many detections reflected surface-level code errors rather than genuine conceptual misunderstandings.

McMiner-in-the-loop tutoring. Instead of batch injection, we integrate McMiner directly into TRAYER’s dialogue loop. After each student response, we (1) generate a diagnostic code completion from the current dialogue context, (2) run McMiner on that code to detect any misconception, and (3) inject only the latest detected misconception into the tutor’s system prompt for the next round. When McMiner outputs NONE, the tutor reverts to its original TRAYER prompt. This round-by-round approach ensures the tutor receives targeted, timely feedback about the student’s current misunderstanding rather than an accumulated list of all past misconceptions. All other settings—turn limit, student model, decoding parameters—are held fixed, and we recompute $\text{pass}@k$ on the resulting student solutions to measure the effect.

Setting	TRAYER	Ours
Student (dialogue)	Mistral-8x7B	Mistral-7B
Student (codegen)	Mistral-8x7B	Llama-3.1-8B
Tutor	Llama-3.1-70B	Llama-3.1-70B
Verifier	Mistral-7B+LoRA	Mistral-7B+LoRA
Repositories	25	4
Tasks	100	22
Quantization	FP16	4-bit NF4

Table 1: Comparison of experimental settings between our reproduction and the original TRAYER paper.

Project	Level	Paper		Ours	
		P@1	P@10	P@1	P@10
sd-forge	low	43.3	100	86.7	100
	med	83.3	100	90.0	100
	high	40.0	100	86.7	100
UHGEval	low	20.0	100	100	100
	med	40.0	100	90.0	100
	high	30.0	100	90.0	100
searcharray	low	3.3	16.7	0.0	0.0
	med	5.0	16.7	0.0	0.0
	high	0.0	0.0	15.0	25.0
EasyVolcap	low	23.3	33.3	23.0	33.3
	med	14.2	25.0	16.0	30.0
	high	23.3	33.3	15.8	33.3

Table 2: Best-round Pass@ k (%) per project and level. Bold indicates the higher value. Our results use the best-performing round for each project-level combination.

5 Experiments and Results

5.1 Experimental Setup

Each task is evaluated at three student proficiency levels: *low_level* (function signature only), *med_level* (+ dependency paths), and *high_level* (+ surrounding code context). For each level, we generate posttest prompts from each of the 8 dialogue rounds and produce $N=10$ code completions per prompt. Pass@ k is computed using the unbiased estimator from Chen et al. (2021).

5.2 Baseline Results

Table 2 shows the best-round Pass@1 and Pass@10 for each project and level, compared to the paper’s reported values on the same repositories.

Simple tasks. On sd-forge and UHGEval, our system outperforms the paper (86–100% vs. 20–83% Pass@1). These tasks are short and well-defined, so minimal tutoring context is often enough; most dialogues stopped after 1–2 rounds.

Complex tasks. On searcharray, we observe 0% Pass@1 at low and med levels across all 8 rounds,

Round	Low	Med	High
R1	12.5	3.3	15.8
R2	6.0	8.0	15.0
R3	4.0	10.0	12.0
R4	0.0	16.0	9.0
R5	15.0	8.0	8.0
R6	16.0	8.0	5.0
R7	23.0	10.0	6.0
R8	21.0	10.0	6.0

Table 3: EasyVolcap Pass@1 (%) by round and level. Low-level students improve with more tutoring (R1: 12.5% → R7: 23.0%), while high-level students degrade (R1: 15.8% → R8: 6.0%).

consistent with the paper. However, at high level, we observe a notable improvement: Pass@1 rises from 0% at R1–R3 to 15% at R8, exceeding the paper’s 0%. This suggests that additional tutoring rounds provide genuine value for complex tasks with sufficient context.

EasyVolcap. Results closely match the paper. At low level, Pass@1 improves from 12.5% (R1) to 23.0% (R7), nearly matching the paper’s 23.3%. At med level, we exceed the paper (16.0% vs. 14.2%). High level trails slightly (15.8% vs. 23.3%) but matches Pass@10.

Aggregate comparison. Averaging across all 22 tasks (using each project-level’s best round), our baseline achieves **27.5% Pass@1** versus the paper’s 20.8% (+6.8pp), with comparable Pass@10 (38.0% vs. 37.9%). The improvement is largest at high level (+9.1pp), suggesting our configuration benefits most when the student has access to richer code context.

5.3 Round-by-Round Analysis

Table 3 illustrates how Pass@1 evolves across tutoring rounds for EasyVolcap, our largest project (12 tasks).

Two opposing trends emerge:

- **Tutoring helps low-level students.** Pass@1 nearly doubles from R1 to R7 (12.5% → 23.0%), as the tutor gradually fills knowledge gaps.
- **Over-tutoring hurts high-level students.** Pass@1 degrades from R1 to R8 (15.8% → 6.0%), suggesting that additional rounds introduce noise or conflicting guidance for students who already have sufficient context.

This pattern motivates our misconception-conditioned approach: rather than providing more tutoring rounds indiscriminately, the tutor should be informed about *what* the student is getting wrong and address it directly.

5.4 McMiner Analysis on Baseline Dialogues

Before integrating McMiner into the tutoring loop, we applied it to 175 code samples across 17 tasks from all four projects using Gemini 2.5 Flash. McMiner detected misconceptions in 142 samples (**81.1%**), with 92.3% at high confidence. Detection rates were uniform across proficiency levels (~80%) but varied by dialogue turn (41.2% at Turn 1, rising to 96.0% by Turn 15), as early turns produce incomplete code that is harder to diagnose. Detected misconceptions clustered into three themes: *tensor/array operations* (53.5%), *OOP/class structure* (30.3%), and *control flow* (16.2%). Per-project detection rates are reported in Appendix A. The high detection rate motivated integrating McMiner directly into the TRAVER tutoring loop.

5.5 McMiner-in-the-Loop Results

We evaluate the effect of conditioning the tutor on McMiner-detected misconceptions. We compare three conditions on EasyVolcap (12 tasks), our largest and most diagnostically rich project:

- **Baseline:** Standard TRAVER tutoring (no misconception information).
- **McMiner:** McMiner-in-the-loop tutoring, where detected misconceptions are injected into the tutor’s system prompt each round.
- **Clean:** Same as McMiner, but misconception-related jargon is stripped from the posttest prompts before code generation, retaining only code-relevant guidance.

EasyVolcap results. Table 4 shows the best-round Pass@1 for each condition and level.

The effect of misconception conditioning is **strongly level-dependent**. At high level, McMiner-Clean achieves 31.7% Pass@1 at R7—a **+15.9 percentage point improvement** over the baseline’s best round of 15.8% (R1). Comparing at the same round (R7), the gap is even larger: the baseline drops to 6.0% while McMiner-Clean reaches 31.7% (+25.7pp), as shown in Table 5.

Level	Baseline	McMiner	Clean
low	23.0 (R7)	11.0 (R6)	10.8 (R1)
med	16.0 (R4)	16.7 (R8)	17.5 (R8)
high	15.8 (R1)	24.2 (R7)	31.7 (R7)

Table 4: EasyVolcap best-round Pass@1 (%) across conditions. McMiner-Clean achieves +15.9pp over the baseline’s best round at high level, but degrades low-level performance by 12pp.

Metric	Baseline	McMiner	Clean	$\Delta\text{BL} \rightarrow \text{CL}$
Pass@1	6.0	24.2	31.7	+25.7
Pass@3	12.2	35.7	40.8	+28.6
Pass@5	15.0	39.6	41.6	+26.6
Pass@10	20.0	41.7	41.7	+21.7

Best-round baseline: 15.8% P@1 (R1) $\rightarrow \Delta = +15.9$

Table 5: EasyVolcap high_level Round 7: Pass@ k (%) comparison. All values are from the same round (R7). The Δ column shows the same-round gap; comparing against the baseline’s best round (15.8% at R1) yields +15.9pp.

However, at low level, both McMiner and Clean **degrade performance** by approximately 12 percentage points relative to the baseline. Both conditions degrade identically, confirming that the damage originates from the tutoring dialogue content itself—not from misconception jargon in the posttest prompts. Low-level students produce code with fundamental errors (e.g., missing imports, wrong function signatures), which McMiner misinterprets as conceptual misconceptions, causing the tutor to focus on the wrong issues.

At med level, the three conditions perform comparably, with Clean showing a marginal edge (+1.5pp over baseline at R8). The clean-prompt ablation provides a small but consistent benefit in early rounds (R1: 7.5% vs. baseline 3.3%) and late rounds (R8: 17.5% vs. baseline 10.0%).

Cross-project comparison. Table 6 shows best-round Pass@1 for all projects under the three conditions.

On sd-forge and UHGEval, McMiner achieves 100% Pass@1 across all levels—matching or exceeding the baseline. These tasks involve short, well-defined functions where misconception-aware tutoring does not interfere with code quality. On searcharray (6 tasks), all McMiner conditions produce 0% Pass@1 across all levels and rounds, compared to the baseline’s 15.0% at high level R8. We also evaluated an extended “Thorough” configuration with semantic deduplication and 12 rounds;

Project	Level	BL	MC	Clean
sd-forge	low	86.7	100	—
	med	90.0	100	—
	high	86.7	100	—
UHGEval	low	100	100	—
	med	90.0	100	—
	high	90.0	100	—
searcharray	low	0.0	0.0	0.0
	med	0.0	0.0	0.0
	high	15.0	11.7*	0.0
EasyVolcap	low	23.0	11.0	10.8
	med	16.0	16.7	17.5
	high	15.8	24.2	31.7

*McMiner-Thorough variant (12 rounds + dedup).

Table 6: Best-round Pass@1 (%) across all projects and conditions. McMiner maintains or improves performance on easy tasks (sd-forge, UHGEval) and improves EasyVolcap at high level, but degrades searcharray and EasyVolcap low level. “—” indicates the Clean ablation was not run.

McMiner-Thorough achieves 11.7% Pass@1 at high level R1 (1/6 tasks), but this remains below the baseline. searcharray tasks require precise knowledge of an unfamiliar API (roaring bitmaps, custom postings lists), where misconception detection provides limited value.

Clean-prompt ablation. The consistent advantage of Clean over raw McMiner on EasyVolcap reveals an important finding about prompt design: misconception-related pedagogical language (e.g., “I notice your code has a fundamental error in how you handle...”) consumes context budget that would otherwise contain actionable code hints. For high-level students who already understand the API surface, removing this meta-discussion and retaining only code-focused guidance yields a +7.5pp improvement over raw McMiner at R7 (31.7% vs. 24.2%).

5.6 Summary of Findings

Table 7 synthesizes our key findings across all conditions. Overall, McMiner-Clean is most effective for high-level students, while low- and medium-level students benefit less from misconception injection. Follow-up experiments suggest that these cases require execution feedback or level-adaptive tutoring rather than additional misconception conditioning.

Level	$\Delta P@1$	Interpretation
low	-12.2	McMiner hurts: noisy code $\rightarrow$ false misconceptions
med	+1.5	Roughly neutral; Clean has marginal edge
high	+15.9	McMiner-Clean excels: targeted guidance works

Table 7: Effect of McMiner-Clean vs. Baseline on EasyVolcap best-round Pass@1 (pp), by student level.

Project	Level	TRAVER		TRAVER + Repair		Δ	
		P@1	P@10	P@1	P@10	P@1	P@10
EasyVolcap	low	23.0	30.0	25.3	40.0	+2.3	+10.0
EasyVolcap	med	16.0	30.0	16.8	37.7	+0.8	+7.7
searcharray	low	0.0	0.0	0.0	0.0	+0.0	+0.0
searcharray	med	0.0	0.0	22.2	33.3 [†]	+22.2	+33.3

Table 8: Post-hoc repair results on selected TRAVER outputs. Repair is applied after TRAVER generation using source context and pytest feedback. [†] For searcharray med-level, repair generated three candidates per task, so the final metric reports P@3 rather than P@10.

5.7 Post-hoc Repair

Tables 8 and 9 summarize our follow-up attempts for low- and medium-level students. Scaffolded TRAVER added static source-code context and smaller tutor steps, but did not improve EasyVolcap low-level performance. Post-hoc repair was more effective: Llama-3.3-70B-Instruct used the original prompt, source context, failed answer, and pytest error output to rewrite only the target function body.

We also tried same-model repair with Llama-3.1-8B-Instruct, matching TRAVER’s code-generation model, but it did not produce reliable gains even with added metadata and recent failure details. Overall, repair improved EasyVolcap low-level P@10 from 30.0 to 40.0, improved EasyVolcap medium-level P@10 from 30.0 to 37.7, and rescued 2 of 6 searcharray medium-level tasks. This suggests that lower-proficiency failures benefit more from execution feedback than from additional static context or misconception injection.

6 Conclusion

We combined misconception detection with dialogue-based tutoring for programming education. Our reproduction of TRAVER on 4 repositories (22 tasks) achieves 27.5% Pass@1 versus the paper’s 20.8% (+6.8pp). Round-by-round analysis reveals that low-level students benefit from extended tutoring while high-level students degrade—

Variant	Setting	Result
TRAVER	EasyVolcap low	baseline
Scaffolded TRAVER	EasyVolcap low	no improvement
TRAVER + Repair	EasyVolcap low	P@10: 30 $\rightarrow$ 40

Table 9: Attempted TRAVER extensions. Scaffolded TRAVER did not improve results. Repair helped using pytest feedback.

motivating our core contribution of integrating McMiner into the tutoring loop.

Our McMiner-in-the-loop experiments show that misconception conditioning is **strongly level-dependent**. At high level, McMiner-Clean achieves 31.7% Pass@1 at R7—a +15.9pp improvement over the baseline’s best round, and a +25.7pp gap at the same round where standard tutoring has degraded to 6.0%. However, the same approach **hurts low-level students** by ~ 12 pp, as McMiner misidentifies syntactic errors as conceptual misconceptions, misdirecting the tutor. Our scaffold and repair ablations further show that lower-proficiency students benefit more from execution feedback than from additional misconception conditioning.

These results suggest the optimal strategy is **level-adaptive**: misconception-aware tutoring should be applied selectively for students with sufficient context. Future work should explore adaptive thresholds that trigger misconception conditioning only when the student’s diagnostic code quality exceeds a minimum threshold.

Limitations

Our evaluation covers only 4 of 25 EvoCodeBench repositories (22 of 100 tasks) because of storage and environment constraints. We also replace the original Mixtral-8x7B student model with Mistral-7B and run local models in 4-bit quantization, which may affect generation quality.

McMiner is evaluated mainly on EasyVolcap: searcharray was difficult for all methods, while sd-forge and UHGEval were mostly solved by the baseline. Broader evaluation is needed to confirm the level-dependent trends.

Acknowledgments

Claude was used to refine the language of the paper and in generation of some scripts.

References

- Erfan Al-Hossami and Razvan Bunescu. 2026. Mcmining: Automated discovery of misconceptions in student code. *arXiv preprint*.
- Neil C. C. Brown and Amjad Altadmri. 2017. [Novice java programming mistakes: Large-scale data vs. educator beliefs](#). *ACM Trans. Comput. Educ.*, 17(2).
- Mark Chen and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Yulin Chen, Ning Ding, Hai-Tao Zheng, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2024. [Empowering private tutoring by chaining large language models](#). In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM '24*, page 354–364, New York, NY, USA. Association for Computing Machinery.
- Nigel J. Francis, Sue Jones, and David P. Smith. 2025. [Generative ai in higher education: Balancing innovation and integrity](#). *British Journal of Biomedical Science*, Volume 81 - 2024.
- W. Lewis Johnson and Elliot Soloway. 1984. Proust: Knowledge-based program understanding. In *Proceedings of the 7th International Conference on Software Engineering, ICSE '84*, page 369–380. IEEE Press.
- Deuksin Kwon, Emily Weiss, Tara Kulshrestha, Kushal Chawla, Gale Lucas, and Jonathan Gratch. 2024. [Are LLMs effective negotiators? systematic evaluation of the multifaceted capabilities of LLMs in negotiation dialogues](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 5391–5413, Miami, Florida, USA. Association for Computational Linguistics.
- et al. Li. 2024. Evocodebench: Benchmarking repository-level code generation. *arXiv preprint*.
- Zewei Tian, Alex Liu, Lief Esbenshade, Shawon Sarkar, Zachary Zhang, Kevin He, and Min Sun. 2025. [Implementation considerations for automated AI grading of student work](#). In *Proceedings of the Artificial Intelligence in Measurement and Education Conference (AIME-Con): Full Papers*, pages 9–20, Wyndham Grand Pittsburgh, Downtown, Pittsburgh, Pennsylvania, United States. National Council on Measurement in Education (NCME).
- Jian Wang and 1 others. 2025. Training turn-by-turn verifiers for dialogue tutoring agents. In *ACL*.
- Rose Wang, Qingyang Zhang, Carly Robinson, Susanna Loeb, and Dorottya Demszky. 2024. [Bridging the novice-expert gap via models of decision-making: A case study on remediating math mistakes](#). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2174–2199, Mexico City, Mexico. Association for Computational Linguistics.

Yu Zhang, Xiusi Chen, Bowen Jin, Sheng Wang, Shuiwang Ji, Wei Wang, and Jiawei Han. 2024. [A comprehensive survey of scientific large language models and their applications in scientific discovery](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8783–8817, Miami, Florida, USA. Association for Computational Linguistics.

Yingxiu Zhao, Bowen Yu, Binyuan Hui, Haiyang Yu, Minghao Li, Fei Huang, Nevin L. Zhang, and Yongbin Li. 2024. [Tree-instruct: A preliminary study of the intrinsic relationship between complexity and alignment](#). In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 16776–16789, Torino, Italia. ELRA and ICCL.

A McMiner Detection Rates

Table 10 shows McMiner’s misconception detection rates on baseline dialogue code, broken down by project.

Project	Samples	Detected	Rate
EasyVolcap	117	102	87.2%
searcharray	51	36	70.6%
xinhua	4	2	50.0%
codeformer	2	2	100%
gfpgan	1	0	0%
Total	175	142	81.1%

Table 10: McMiner misconception detection rates on baseline dialogue code, by project.

B Per-Project Round-by-Round Results

This appendix provides complete Pass@ k ($k \in \{1, 3, 5, 10\}$) results for all projects, levels, and tutoring rounds. BL = Baseline, MC = McMiner-Loop, CL = McMiner-Clean.

B.1 sd-forge (3 tasks) and UHGEval (1 task)

These projects are essentially solved: all conditions achieve near-perfect scores. Table 11 reports the baseline alongside McMiner-Loop results from the corrected Colab evaluation.

Analysis. These four tasks—codeformer_model.setup_model (model initialization), gfpgan_model.gfpgan_fix_faces and gfpgan_model.setup_model (face restoration setup), and xinhua.XinhuaHallucinations.statistics (JSON aggregation)—involve short, well-defined functions (1–10 lines) with clear signatures and

Proj	Level	Cond.	P@1	P@3	P@5	P@10
codeformer	low	BL R1	90.0	100	100	100
		MC R1–3	100	100	100	100
	med	BL R1	100	100	100	100
		MC R1–3	100	100	100	100
	high	BL R1	90.0	100	100	100
		MC R1–3	100	100	100	100
gfpagan	low	BL R1	85.0	100	100	100
		MC R1–3	95–100	100	100	100
	med	BL R1	85.0	100	100	100
		MC R1–3	100	100	100	100
	high	BL R1	85.0	100	100	100
		MC R1–8	100	100	100	100
xinhua	low	BL R1	100	100	100	100
		MC R1–3	100	100	100	100
	med	BL R1	90.0	100	100	100
		MC R1–3	100	100	100	100
	high	BL R1	90.0	100	100	100
		MC R1–3	100	100	100	100

Table 11: sd-forge and UHGEval Pass@ k (%). McMiner matches or exceeds baseline across all rounds (all rounds identical; range shown). BL early-stopped at R1–R3; MC continued for 3–8 rounds.

minimal API surface. The baseline already achieves 85–100% P@1 at R1, indicating that even a single tutoring round provides sufficient context for the student model to generate correct completions. McMiner maintains or slightly improves these scores: misconception-aware tutoring neither helps nor harms when the underlying task is simple enough that the code generation model can solve it from the function signature alone. The baseline conversations early-stop after 1–3 rounds because the tutor recognizes the student has converged, while McMiner conversations extend to 3–8 rounds due to the misconception injection prompting additional dialogue turns. Despite these extra rounds, performance remains saturated at 100%, confirming that McMiner does not introduce harmful noise for easy tasks.

B.2 searcharray (6 tasks)

B.2.1 Baseline (R1–R8)

Low and med levels produce 0% on all metrics across all 8 rounds. Table 12 shows the high_level detail.

Baseline analysis. searcharray is the hardest project in our benchmark. Its 6 tasks require knowledge of roaring bitmaps, custom postings lists, and the searcharray library’s internal API—none of which appear in standard training data. At low and med levels, no task passes across any round: the simulated students produce code with

Round	P@1	P@3	P@5	P@10
R1	0.0	0.0	0.0	0.0
R2	0.0	0.0	0.0	0.0
R3	0.0	0.0	0.0	0.0
R4	7.5	17.7	22.9	25.0
R5	0.0	0.0	0.0	0.0
R6	12.5	22.9	24.9	25.0
R7	7.5	17.7	22.9	25.0
R8	15.0	24.2	25.0	25.0

Table 12: searcharray baseline high_level Pass@ k (%). The only task that passes is searcharray.solr.edismax. Low and med = 0% for all rounds.

wrong attribute names (e.g., positions_dict instead of the correct accessor), import nonexistent libraries (anserini), or trigger infinite loops in while-based bit counting. At high level, a single task (searcharray.solr.edismax) begins passing at R4, reaching 15% P@1 by R8. This task involves query parsing logic that is more amenable to iterative refinement through tutoring. The intermittent zero at R5 (between non-zero R4 and R6) reflects the stochastic nature of code generation: the model occasionally regresses before recovering in subsequent rounds. The gap between P@1 (15%) and P@10 (25%) at R8 indicates that correct solutions exist in the completion distribution but are not consistently the top-ranked output.

B.2.2 McMiner-Thorough (R1–R12)

Low and med levels remain 0% for all 12 rounds. Table 13 shows the high_level detail.

Round	P@1	P@3	P@5	P@10
R1	11.7	16.5	16.7	16.7
R2	5.0	11.8	15.3	16.7
R3–R12	0.0	0.0	0.0	0.0

Table 13: searcharray McMiner-Thorough high_level Pass@ k (%). Best result is 11.7% P@1 at R1 (1/6 tasks), below baseline’s 15.0% at R8.

McMiner-Thorough analysis. The Thorough configuration applies semantic deduplication to misconception prompts and extends tutoring to 12 rounds with accumulated context. Despite these enhancements, searcharray remains largely unsolved. The best result—11.7% P@1 at high_level R1—solves the same single task (edismax) as the baseline, but with lower probability. Critically, performance degrades to 0% by R3 and never recovers through R12, whereas the baseline improves from

R4 to R8. This suggests that McMiner’s misconception injection actively interferes with the tutoring trajectory for API-heavy tasks: the tutor spends dialogue turns addressing perceived conceptual misunderstandings when the real bottleneck is the student model’s lack of domain-specific API knowledge. Extended rounds (R9–R12) provide no recovery, confirming that more tutoring cannot compensate for fundamentally misaligned misconception feedback.

B.3 EasyVolcap (12 tasks) — Full 3-Way Comparison

B.3.1 low_level

Low-level analysis. McMiner consistently degrades low_level performance, with the gap widening as rounds increase. At R1, the three conditions are nearly identical (12.5/10.8/10.8% P@1), but by R7 the baseline reaches 23.0% while both McMiner (10.0%) and Clean (8.0%) plateau around 10%. This divergence pattern is consistent across all Pass@k values: at R7, the baseline achieves 27.1/29.2/30.0% (P@3/5/10) while McMiner and Clean are stuck at 10%. The root cause is that low-level students produce syntactically broken diagnostic code that McMiner misinterprets as conceptual misconceptions. For example, a student who writes `self.data = None` (a placeholder) triggers a misconception about “fundamental misunderstanding of data initialization,” causing the tutor to spend subsequent rounds correcting a non-existent conceptual error rather than guiding the student toward the correct API usage. Crucially, the Clean ablation provides no benefit over raw McMiner at this level (~1pp difference), confirming that the damage originates in the *tutoring dialogue content itself*, not in residual misconception jargon in the posttest prompt. The baseline’s R7 peak (23.0%) followed by slight decline at R8 (21.0%) suggests an optimal tutoring window of 6–7 rounds for low-level students.

B.3.2 med_level

Med-level analysis. The med_level results exhibit a distinctive two-phase pattern. In *Phase 1* (R1–R5), the baseline generally outperforms McMiner conditions, with the baseline peaking at 16.0% P@1 at R4 while McMiner collapses to 0.0% and Clean to 1.7% at the same round. The R4 collapse is particularly striking: it coincides with the point where McMiner misconception feedback accumulates enough to derail the tutoring trajec-

tory, while the baseline tutor continues productive instruction. In *Phase 2* (R6–R8), the relationship inverts: McMiner conditions recover and surpass the baseline, with Clean reaching 17.5% P@1 at R8 versus the baseline’s 10.0%. This recovery suggests that after enough rounds, the accumulated tutoring context overcomes the initial misconception-induced noise. Clean consistently outperforms raw McMiner by 0.8–4.2pp at P@1 across most rounds, with the advantage most pronounced at R1 (7.5% vs. 2.5%). This pattern holds across all Pass@k: at R1, Clean achieves 17.3/22.3/25.0% (P@3/5/10) versus McMiner’s 6.9/10.6/16.7%, indicating that removing misconception jargon from the posttest prompt frees context budget for code-relevant hints. The net effect is roughly neutral: baseline best (16.0% at R4) versus Clean best (17.5% at R8) yields a modest +1.5pp gain.

B.3.3 high_level

High-level analysis. This is our headline finding. The high_level results show a dramatic and consistent advantage for McMiner conditions, with the effect amplifying through later rounds. The baseline peaks at R1 (15.8% P@1) and degrades steadily to 6.0% by R7—a pattern we attribute to “over-tutoring,” where additional dialogue rounds introduce noise and conflicting guidance for students who already understand the API surface. McMiner reverses this trajectory: performance *improves* from 15.8% (R1) to 24.2% (R7), a +8.4pp gain over the same-round baseline. Clean amplifies this further to 31.7% at R7, yielding a +25.7pp same-round delta.

The Clean advantage over raw McMiner (31.7% vs. 24.2% at R7, +7.5pp) reveals an important prompt engineering insight: misconception-related pedagogical language (e.g., “I notice your code has a fundamental error in how you handle...”) consumes the 60-word posttest prompt budget. For high-level students who already understand the concepts, this meta-discussion displaces actionable code hints. Stripping it leaves more room for concrete API guidance that the code generation model can directly leverage.

The improvement is consistent across all Pass@k. At R7, Clean achieves 40.8% P@3, 41.6% P@5, and 41.7% P@10, versus baseline’s 12.2/15.0/20.0%. The narrowing gap between P@1 and P@10 for Clean (31.7% vs. 41.7%) compared to baseline (6.0% vs. 20.0%) indicates that McMiner-Clean not only increases the probabil-

Rd	Pass@1			Pass@3			Pass@5			Pass@10		
	BL	MC	CL	BL	MC	CL	BL	MC	CL	BL	MC	CL
R1	12.5	10.8	10.8	22.2	15.3	17.8	27.1	19.0	23.1	33.3	25.0	33.3
R2	6.0	4.0	5.0	15.4	8.3	11.3	21.9	9.8	14.8	30.0	10.0	20.0
R3	4.0	7.0	6.0	10.1	17.2	15.4	14.2	23.3	21.9	20.0	30.0	30.0
R4	0.0	2.0	2.0	0.0	5.3	5.3	0.0	7.8	7.8	0.0	10.0	10.0
R5	15.0	7.0	8.0	19.7	9.9	10.0	20.0	10.0	10.0	20.0	10.0	10.0
R6	16.0	11.0	10.0	20.0	13.0	10.0	20.0	15.0	10.0	20.0	20.0	10.0
R7	23.0	10.0	8.0	27.1	10.0	10.0	29.2	10.0	10.0	30.0	10.0	10.0
R8	21.0	5.0	4.0	23.0	9.2	8.3	25.0	10.0	9.8	30.0	10.0	10.0

Table 14: EasyVolcap low_level: full Pass@ k (%) for all conditions. Baseline dominates consistently from R5 onward across all metrics.

Rd	Pass@1			Pass@3			Pass@5			Pass@10		
	BL	MC	CL	BL	MC	CL	BL	MC	CL	BL	MC	CL
R1	3.3	2.5	7.5	8.9	6.9	17.3	13.0	10.6	22.3	16.7	16.7	25.0
R2	8.0	5.0	8.3	16.2	11.4	15.0	19.1	14.6	16.5	20.0	16.7	16.7
R3	10.0	5.0	3.3	15.3	8.1	6.9	17.8	8.3	8.1	20.0	8.3	8.3
R4	16.0	0.0	1.7	23.7	0.0	4.4	27.5	0.0	6.5	30.0	0.0	8.3
R5	8.0	2.5	4.2	15.7	5.9	7.6	20.0	7.6	8.3	30.0	8.3	8.3
R6	8.0	7.5	7.5	10.0	8.3	8.3	10.0	8.3	8.3	10.0	8.3	8.3
R7	10.0	15.0	16.7	10.0	16.7	16.7	10.0	16.7	16.7	10.0	16.7	16.7
R8	10.0	16.7	17.5	13.0	16.7	19.2	15.0	16.7	20.8	20.0	16.7	25.0

Table 15: EasyVolcap med_level: full Pass@ k (%) for all conditions. Roughly neutral; Clean has an edge at R1 and R7–R8.

ity of correct solutions but also concentrates them toward the top of the generation distribution.

The R8 dip in Clean (15.0% vs. McMiner’s 21.7%) is likely attributable to variance at the tutoring ceiling: by R8, most improvable tasks have already been solved, and the remaining unsolved tasks are resistant to further tutoring.

C Scaffold and Repair Ablations

We ran additional ablations to understand why low-level and API-heavy tasks remained difficult. These experiments tested whether better structure, same-model feedback, or stronger post-hoc debugging could improve TRAVER outputs.

Scaffolded TRAVER. Scaffolded TRAVER was a prompt-level intervention for low-level students. We added static structural guidance such as target-function context, smaller implementation steps, and skeleton-style hints. This did not improve EasyVolcap low-level performance. The likely reason is that the main bottleneck was not task structure alone, but missing project-specific API knowledge. Extra scaffold text often increased prompt noise without giving the model enough information to select the correct EasyVolcap operations.

Same-model repair. We tested whether execution feedback alone could improve failed TRAVER

completions. Repair was applied only to failed outputs; originally passing completions were preserved. Using the same Llama-3.1-8B model as the code generator, repair rescued no additional EasyVolcap low-level tasks. A peer-repair variant, where the same model saw multiple failed attempts, also rescued none. This suggests that the 8B model could see the pytest failure but could not reliably turn it into correct project-specific code.

Second-student prompting. We also simulated a second student with the same model capacity. The prompt included task metadata, previous failed attempts, and failure summaries, but no reference solution. This produced 0/12 on EasyVolcap low-level and 0/10 on EasyVolcap med-level. Prior failed attempts appeared to pollute the prompt rather than help the model recover.

Ad-hoc stronger-model repair. Finally, we tested a stronger post-hoc repair setting. TRAVER generated code normally, but failed completions were passed to a 70B repair model with the original prompt, source context, failed body, and pytest output. This improved EasyVolcap low-level from 3/12 to 5/12 solved tasks. However, this changes model capacity, so the result should be interpreted as a practical debugging extension rather than evidence that feedback alone improves the original

Rd	Pass@1			Pass@3			Pass@5			Pass@10		
	BL	MC	CL	BL	MC	CL	BL	MC	CL	BL	MC	CL
R1	15.8	15.8	19.2	25.0	28.0	28.8	28.5	31.4	31.4	33.3	33.3	33.3
R2	15.0	13.3	16.7	25.4	26.0	26.0	28.9	30.8	29.0	30.0	33.3	33.3
R3	12.0	15.0	10.0	20.1	24.9	17.2	24.2	29.6	21.3	30.0	33.3	25.0
R4	9.0	15.0	16.7	16.8	21.1	23.5	19.2	23.1	24.8	20.0	25.0	25.0
R5	8.0	23.3	22.5	12.9	27.4	31.2	15.0	29.2	33.1	20.0	33.3	33.3
R6	5.0	21.7	22.5	11.3	29.2	28.7	14.8	31.5	31.4	20.0	33.3	33.3
R7	6.0	24.2	31.7	12.2	35.7	40.8	15.0	39.6	41.6	20.0	41.7	41.7
R8	6.0	21.7	15.0	9.7	24.7	19.1	10.0	25.0	20.8	10.0	25.0	25.0

Table 16: EasyVolcap high_level: full Pass@ k (%) for all conditions. McMiner-Clean dominates from R4 onward across all metrics. At R7, Clean achieves 31.7%/40.8%/41.6%/41.7% (P@1/3/5/10) vs. baseline’s 6.0%/12.2%/15.0%/20.0%.

Variant	Setting	Result
TRAVER baseline	EasyVolcap low	3/12 solved
Scaffolded TRAVER	EasyVolcap low	no improvement
8B same-model repair	EasyVolcap low	0 rescued; remains 3/12
8B peer repair	EasyVolcap low	0 rescued; remains 3/12
8B second-student prompt	EasyVolcap low/med	0/12 low, 0/10 med
70B post-hoc repair	EasyVolcap low	improves to 5/12

Table 17: Scaffold and repair ablations. Repair variants preserve originally passing TRAVER completions and only attempt to rescue failed tasks.

8B generator.